Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

Source Code:
`https://github.com/Hasini923/Deep_Learning/tree/main/single_layer_perceptron`

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Output:

```
First Five Samples
   Variance  Skewness  Curtosis  Entropy  Class
0   3.62160    8.6661   -2.8073 -0.44699      0
1   4.54590    8.1674   -2.4586 -1.46210      0
2   3.86600   -2.6383    1.9242  0.10645      0
3   3.45660    9.5228   -4.0112 -3.59440      0
4   0.32924   -4.4552    4.5718 -0.98880      0

Dataset Shape:
(1372, 5)

Missing Values:
Variance    0
Skewness    0
Curtosis    0
Entropy     0
Class       0

Descriptive Statistics:
          Variance     Skewness     Curtosis      Entropy        Class
count  1372.000000  1372.000000  1372.000000  1372.000000  1372.000000
mean      0.433735     1.922353     1.397627    -1.191657     0.444606
std       2.842763     5.869047     4.310030     2.101013     0.497103
min      -7.042100   -13.773100    -5.286100    -8.548200     0.000000
25%      -1.773000    -1.708200    -1.574975    -2.413450     0.000000
50%       0.496180     2.319650     0.616630    -0.586650     0.000000
75%       2.821475     6.814625     3.179250     0.394810     1.000000
max       6.824800    12.951600    17.927400     2.449500     1.000000
```

Figure 1: first five samples, shape, missing values and descriptive statistics.

Inference: *The dataset has 1372 banknote samples with 4 features and no missing values, so it's clean and ready to use straightaway. The feature values are spread across quite different ranges.*

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Output:

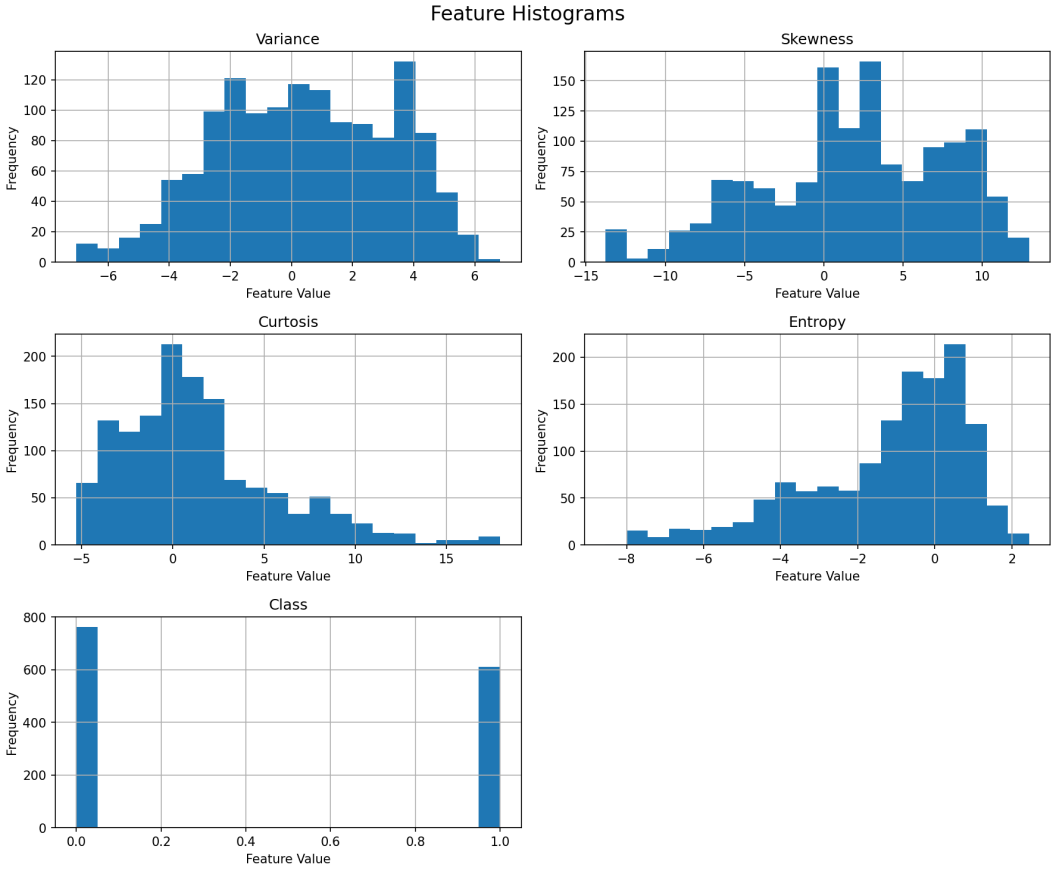

Figure 2: Feature histograms for Variance, Skewness, Curtosis and Entropy.

Inference: *Skewness and Curtosis look noticeably skewed with a long tail on one side, while Entropy is closer to a normal-ish shape.*

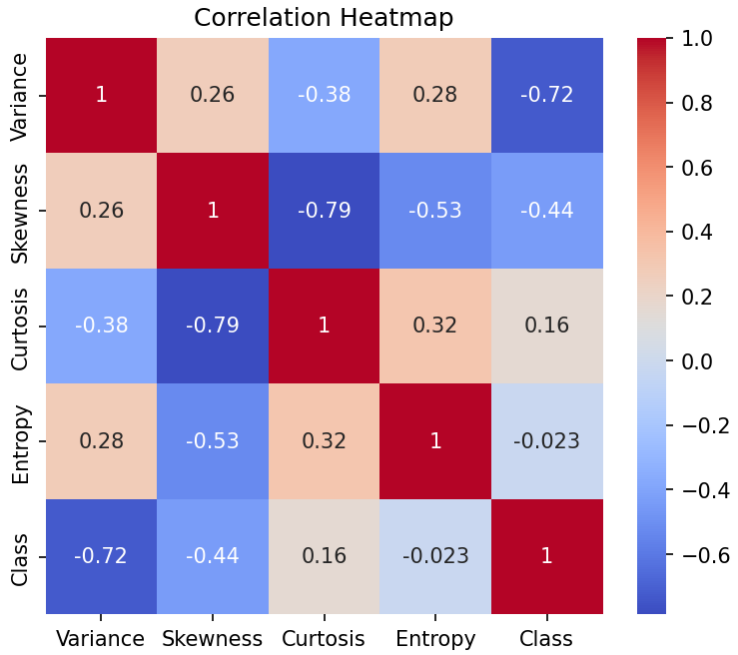

Figure 3: Correlation heatmap of the four numerical features.

Inference: *Variance and Skewness show the strongest relationship with the target class, whereas entropy barely correlates with anything.*

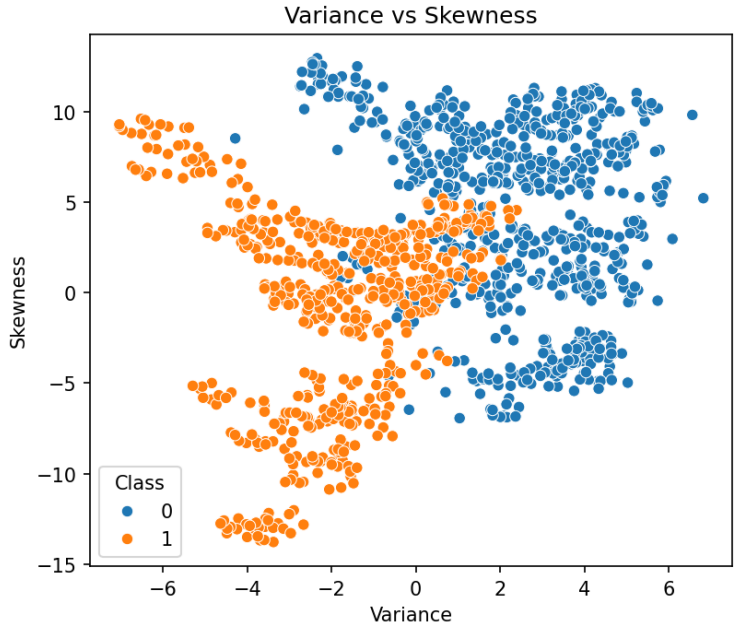

Figure 4: Scatter plot of Variance vs Skewness colored by class.

Inference: *The two classes form distinct clusters in this scatter plot, with only a small overlapping region in the middle. This is good for a perceptron since it works best when the data is almost linearly separable.*

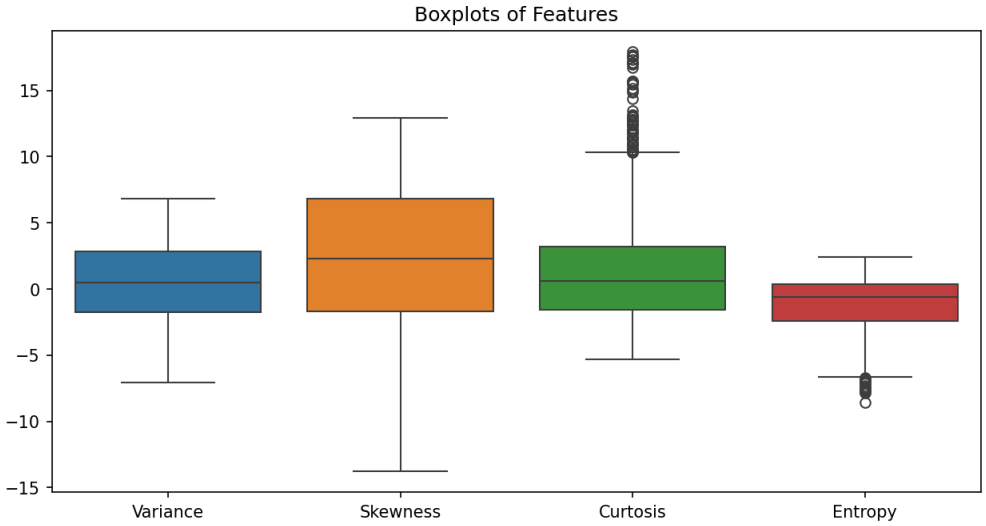

Figure 5: Boxplots of all four features before normalization.

Inference: *The boxplots show that the features sit on completely different scales, and some outliers stay above and below the whiskers.*

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Output:

```
Training Samples: 1097
Testing Samples: 275
```

Figure 6: Number of training and testing samples after the 80:20 split.

Inference: *The split gives 1097 samples for training and 275 for testing, which lines up correctly with the 80:20 ratio. The labels were also switched from {0,1} to {-1,+1} because of the perceptron's weight update rule.*

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization

- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Output:

```
Epoch 1
Misclassified Samples: 166
Updated Weights: [-0.68274078 -0.38668532 -0.53190972  0.15293538]
Updated Bias: 0.7999999999999999
---------------------------------------
Epoch 2
Misclassified Samples: 88
Updated Weights: [-0.83116105 -0.61463152 -0.76878383  0.11624225]
Updated Bias: 0.9999999999999999
---------------------------------------
Epoch 3
Misclassified Samples: 57
Updated Weights: [-0.87315739 -0.65655397 -0.95930508  0.15060017]
Updated Bias: 1.0999999999999999
---------------------------------------
Epoch 4
Misclassified Samples: 51
Updated Weights: [-0.95385683 -0.79743107 -0.9285397   0.10170151]
Updated Bias: 1.2
---------------------------------------
Epoch 5
Misclassified Samples: 47
Updated Weights: [-0.97783436 -0.85640299 -1.00255103  0.05781127]
Updated Bias: 1.3
---------------------------------------
Epoch 6
Misclassified Samples: 45
Updated Weights: [-1.05811604 -0.88618353 -1.01493801  0.04330288]
Updated Bias: 1.4000000000000001
---------------------------------------
Epoch 7
Misclassified Samples: 48
Updated Weights: [-1.11600397 -0.99126352 -1.08300179  0.09091242]
Updated Bias: 1.4000000000000001
---------------------------------------
Epoch 8
Misclassified Samples: 32
Updated Weights: [-1.17964956 -1.01244097 -1.16160002  0.0999082 ]
Updated Bias: 1.4000000000000001
---------------------------------------

...

Epoch 33
Misclassified Samples: 20
Updated Weights: [-1.63917504 -1.54802555 -1.78330421  0.06353891]
Updated Bias: 2.1000000000000005
---------------------------------------
Epoch 34
Misclassified Samples: 20
Updated Weights: [-1.65772525 -1.60401495 -1.76746558  0.05764768]
Updated Bias: 2.1000000000000005
---------------------------------------
Epoch 35
Misclassified Samples: 16
Updated Weights: [-1.63449677 -1.61717053 -1.81053737  0.10981736]
Updated Bias: 2.1000000000000005
---------------------------------------
Epoch 36
Misclassified Samples: 20
Updated Weights: [-1.66515936 -1.64595102 -1.80655822  0.08477303]
Updated Bias: 2.1000000000000005
---------------------------------------
Epoch 37
Misclassified Samples: 27
Updated Weights: [-1.6567789  -1.56403105 -1.85196683  0.01561381]
Updated Bias: 2.2000000000000006
---------------------------------------
Epoch 38
Misclassified Samples: 24
Updated Weights: [-1.73443389 -1.66032174 -1.78778905  0.08117577]
Updated Bias: 2.2000000000000006
---------------------------------------
Epoch 39
```

Inference: *The error count is high in the first couple of epochs but drops quickly after that, showing the model is learning fast early on. By the later epochs the misclassification count settles down to a small, steady number, which means the perceptron has more or less converged.*

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Output:

```
Accuracy : 0.9854545454545455
Precision: 1.0
Recall   : 0.968503937007874
F1 Score : 0.984

Confusion Matrix
[[148   0]
 [  4 123]]
```

Figure 8: Accuracy, Precision, Recall, F1-score and raw confusion matrix values on the test set.

Inference: *The model performs with accuracy above 98% and strong precision, recall and F1-scores to match. Looking at the confusion matrix, the mistakes are almost all in one direction, so the model leans slightly toward one type of error.*

Additionally, the confusion matrix is visualized as a heatmap:

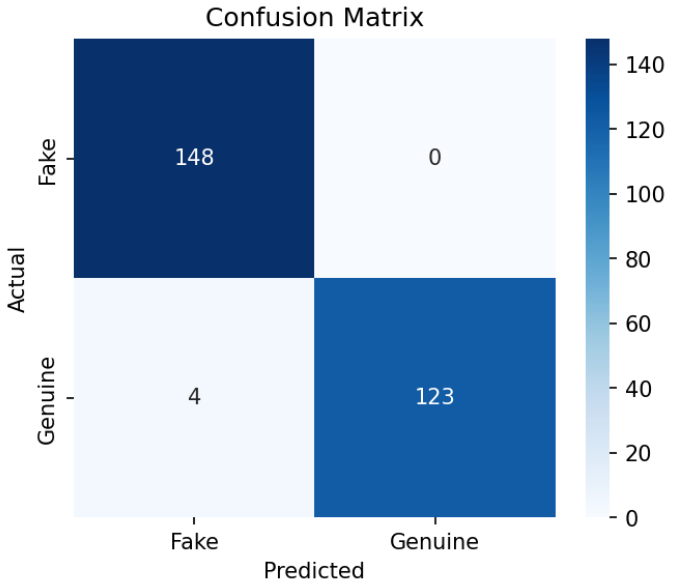

Figure 9: Confusion matrix heatmap for the test set.

Inference: *Out of 275 test notes, 148 fakes and 123 genuine notes were classified correctly, with only 4 genuine notes wrongly flagged as fake and zero fakes let through as genuine.*

7. Mandatory Plots

Plot	**Purpose**	**Expected Inference**
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.

Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot. The Feature Histograms, Correlation Heatmap, Scatter Plot and Confusion Matrix are already shown above under Tasks 2 and 6. The remaining plots are shown below.

Training Error vs Epoch

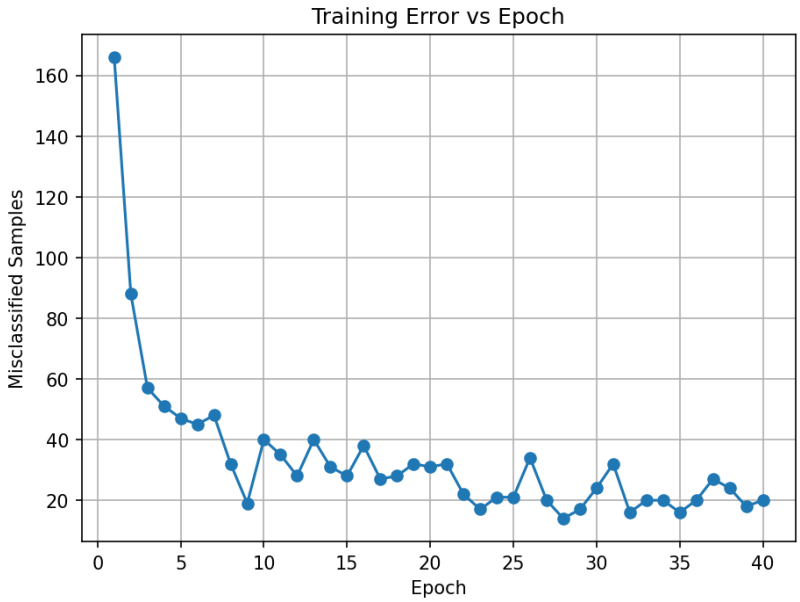

Figure 10: Training error (misclassified samples) versus epoch.

Inference: *The error curve drops sharply in the first few epochs and then flattens out close to zero. This confirms that the banknote data is close enough to linearly separable for the perceptron to learn a good boundary.*

Weight Evolution

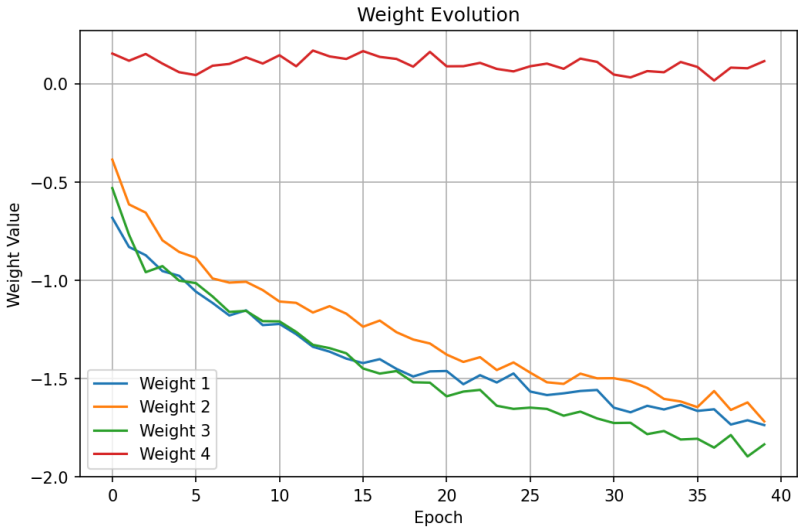

Figure 11: Evolution of each weight across training epochs.

Inference: *The weights swing around a bit early in training but settle into stable values after a while, and variance ends up with the largest magnitude.*

Bias Evolution

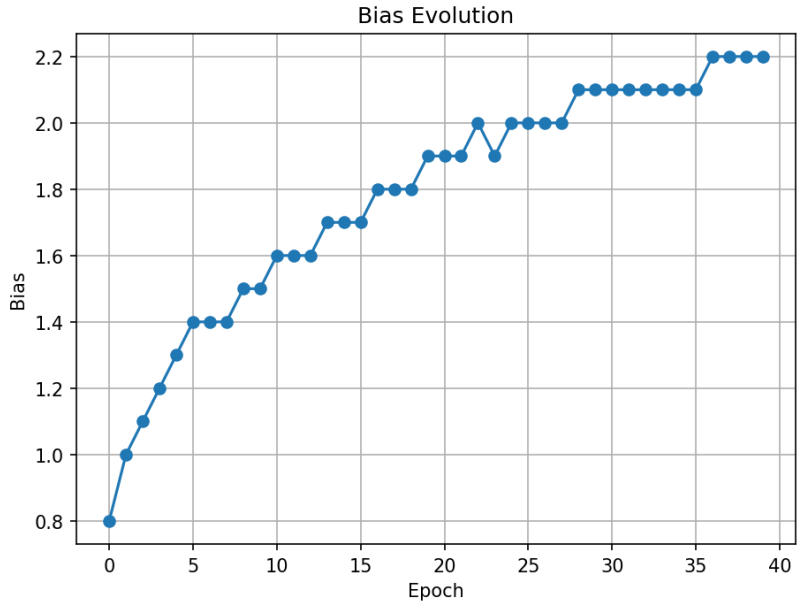

Figure 12: Evolution of the bias term across training epochs.

Inference: *The bias moves around a bit in the early epochs as the model adjusts, then flattens out around the same time the weights do. This happens because the bias and weights are updated together every time a sample is misclassified.*

Learning Rate Comparison

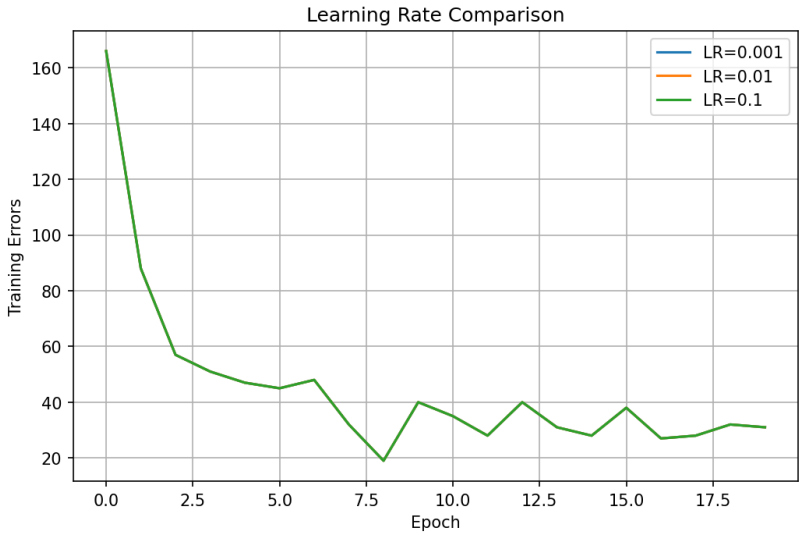

Figure 13: Training error curves for learning rates 0.001, 0.01 and 0.1.

Inference: *All three learning rates actually produce the exact same error curve, which is why only one line is visible. This happens because the perceptron's prediction only depends on the sign of the weighted sum, and scaling every weight update by a constant learning rate doesn't change that sign, so the model misclassifies the same samples at the same epochs regardless of the learning rate.*

Decision Boundary (Optional)

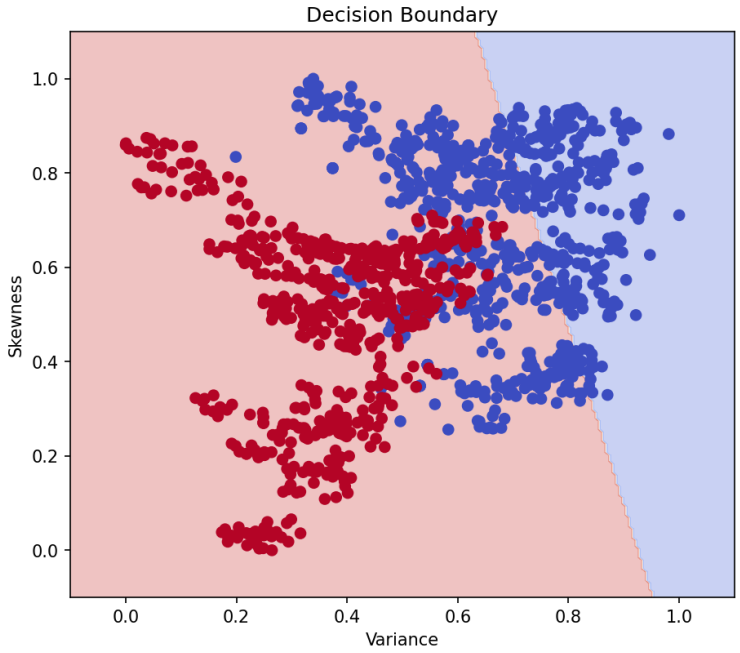

Figure 14: Decision boundary of the perceptron using Variance and Skewness only.

Inference: *Using just Variance and Skewness, the decision boundary still splits the two classes,*

with only a small number of points falling on the wrong side. This shows these two features alone carry most of the information needed to tell fake and genuine notes apart.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80 : 20
Learning Rate	0.1
Epochs	40
Final Weights	[-1.7373, -1.7186, -1.8354, 0.1144]
Final Bias	2.2
Accuracy	0.9855
Precision	1.0
Recall	0.9685
F1-score	0.984

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	166	-0.6827	-0.3867	0.80
10	40	-1.2286	-1.0505	1.50
20	31	-1.4641	-1.3221	1.90
30	24	-1.5586	-1.4994	2.10
40	20	-1.7373	-1.7186	2.20

9. Additional Tasks

9.1 Learning Rate Repetition (0.001, 0.01, 0.1)

Output:

```
================================================
Training Perceptron with Learning Rate = 0.001
================================================
Epoch 1
Misclassified Samples: 166
Updated Weights: [-0.00682741 -0.00386685 -0.0053191   0.00152935]
Updated Bias: 0.008
----------------------------------------
Epoch 2
Misclassified Samples: 88
Updated Weights: [-0.00831161 -0.00614632 -0.00768784  0.00116242]
Updated Bias: 0.010000000000000002
----------------------------------------
Epoch 3
Misclassified Samples: 57
Updated Weights: [-0.00873157 -0.00656554 -0.00959305  0.001506  ]
Updated Bias: 0.011000000000000003
----------------------------------------
...
Epoch 38
Misclassified Samples: 24
Updated Weights: [-0.01734434 -0.01660322 -0.01787789  0.00081176]
Updated Bias: 0.022000000000000013
----------------------------------------
Epoch 39
Misclassified Samples: 18
Updated Weights: [-0.01712691 -0.01621979 -0.01896825  0.00077931]
Updated Bias: 0.022000000000000013
----------------------------------------
Epoch 40
Misclassified Samples: 20
Updated Weights: [-0.01737314 -0.01718642 -0.01835368  0.00114368]
Updated Bias: 0.022000000000000013
----------------------------------------

================================================
Training Perceptron with Learning Rate = 0.01
================================================
Epoch 1
Misclassified Samples: 166
Updated Weights: [-0.06827408 -0.03866853 -0.05319097  0.01529354]
Updated Bias: 0.08
----------------------------------------
Epoch 2
Misclassified Samples: 88
Updated Weights: [-0.0831161  -0.06146315 -0.07687838  0.01162422]
Updated Bias: 0.09999999999999999
----------------------------------------
Epoch 3
Misclassified Samples: 57
Updated Weights: [-0.08731574 -0.0656554  -0.09593051  0.01506002]
Updated Bias: 0.10999999999999999
----------------------------------------
...
Epoch 38
Misclassified Samples: 24
Updated Weights: [-0.17344339 -0.16603217 -0.1787789   0.00811758]
Updated Bias: 0.22000000000000006
----------------------------------------
Epoch 39
Misclassified Samples: 18
Updated Weights: [-0.17126906 -0.16219786 -0.1896825   0.00779308]
Updated Bias: 0.22000000000000006
----------------------------------------
Epoch 40
Misclassified Samples: 20
Updated Weights: [-0.17373145 -0.17186415 -0.18353676  0.01143681]
Updated Bias: 0.22000000000000006
----------------------------------------

================================================
Training Perceptron with Learning Rate = 0.1
================================================
Epoch 1
Misclassified Samples: 166
Updated Weights: [-0.68274078 -0.38668532 -0.53190972  0.15293538]
Updated Bias: 0.7999999999999999
----------------------------------------
Epoch 2
Misclassified Samples: 88
Updated Weights: [-0.83116105 -0.61463152 -0.76878383  0.11624225]
Updated Bias: 0.9999999999999999
```

Inference: *Looking at the printed logs, the smallest learning rate (0.001) is learning slowly. The larger rates (0.01 and 0.1) bring the error count down to a low, steady value much earlier in training.*

	Learning Rate	Accuracy	Precision	Recall	F1 Score
0	0.001	0.985455	1.0	0.968504	0.984
1	0.010	0.985455	1.0	0.968504	0.984
2	0.100	0.985455	1.0	0.968504	0.984

Figure 16: Comparison table of Accuracy, Precision, Recall and F1-score across learning rates.

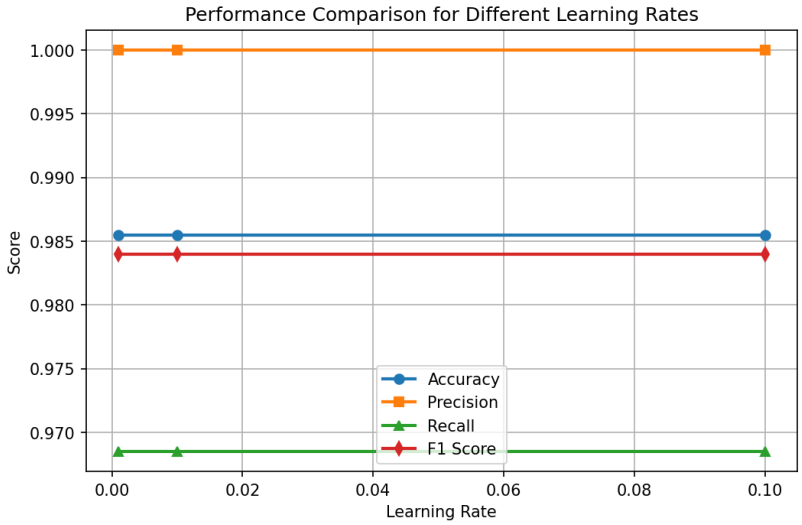

Figure 17: Accuracy, Precision, Recall and F1-score plotted against learning rate.

Inference: *Accuracy, precision, recall and F1 all follow a similar trend across the three learning rates rather than moving independently. None of the metrics swing wildly, which just tells us this dataset is forgiving enough that the model doesn't need super precise tuning to perform well.*

```
Learning Rate: 0.001
Weights: [-0.01737314 -0.01718642 -0.01835368  0.00114368]
Bias: 0.022000000000000013

Learning Rate: 0.01
Weights: [-0.17373145 -0.17186415 -0.18353676  0.01143681]
Bias: 0.22000000000000006

Learning Rate: 0.1
Weights: [-1.73731449 -1.71864154 -1.83536757  0.11436807]
Bias: 2.2000000000000006
```

Figure 18: Final weights and bias obtained for each learning rate.

Inference: *The final weights differ a bit in scale between the three learning rates, since a larger learning rate takes bigger steps and can land on a different point along the separating boundary. But, the overall direction of the weight vector of which feature matters most stays roughly the same across all three, so the differences are more about scale than substance.*

9(a) Single Layer Perceptron: OR, AND and NOT Gates

Question: Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.

Output:

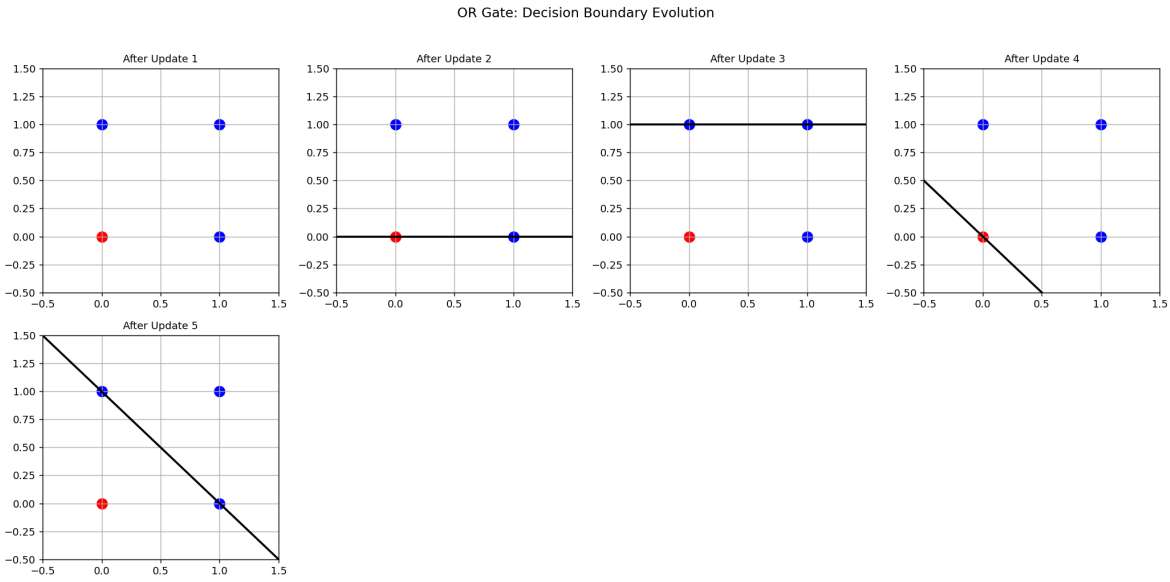

Figure 19: Decision boundary evolution for the OR Gate after each of the 5 weight updates.

Inference: *The OR gate converges in just 5 updates (3 epochs), landing on weights [0.10, 0.10] and bias -0.10.*

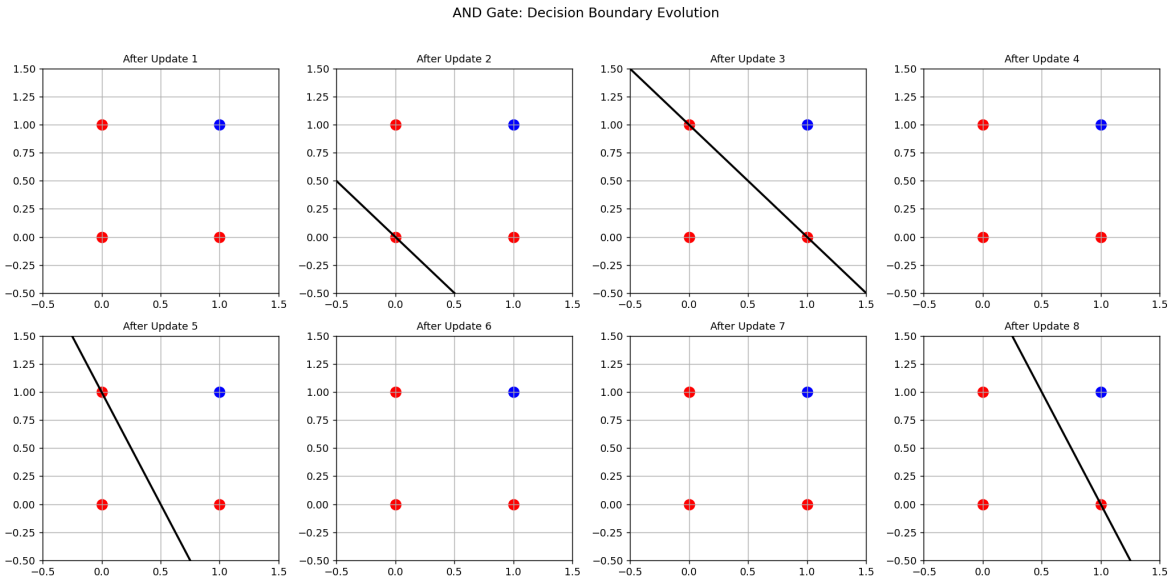

Figure 20: Decision boundary evolution for the AND Gate after each of the 8 weight updates.

Inference: *AND takes a bit longer (8 updates) since only one of four points is class 1, converging to weights [0.20, 0.10] and bias -0.20.*

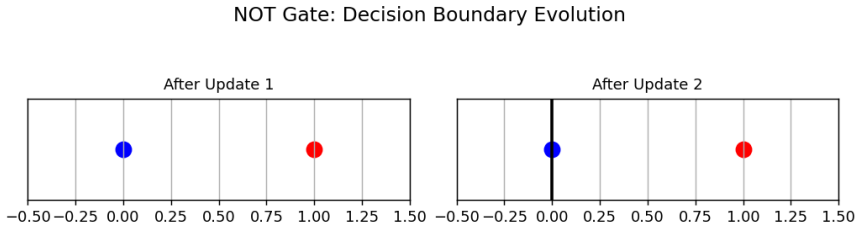

Figure 21: Decision boundary evolution for the NOT Gate after each of the 2 weight updates.

Inference: *NOT converges in only 2 updates, with weight -0.10 and bias 0.00 placing the threshold cleanly between 0 and 1.*

9(b) Single Layer Perceptron: XOR Gate

Question: Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.

Output:

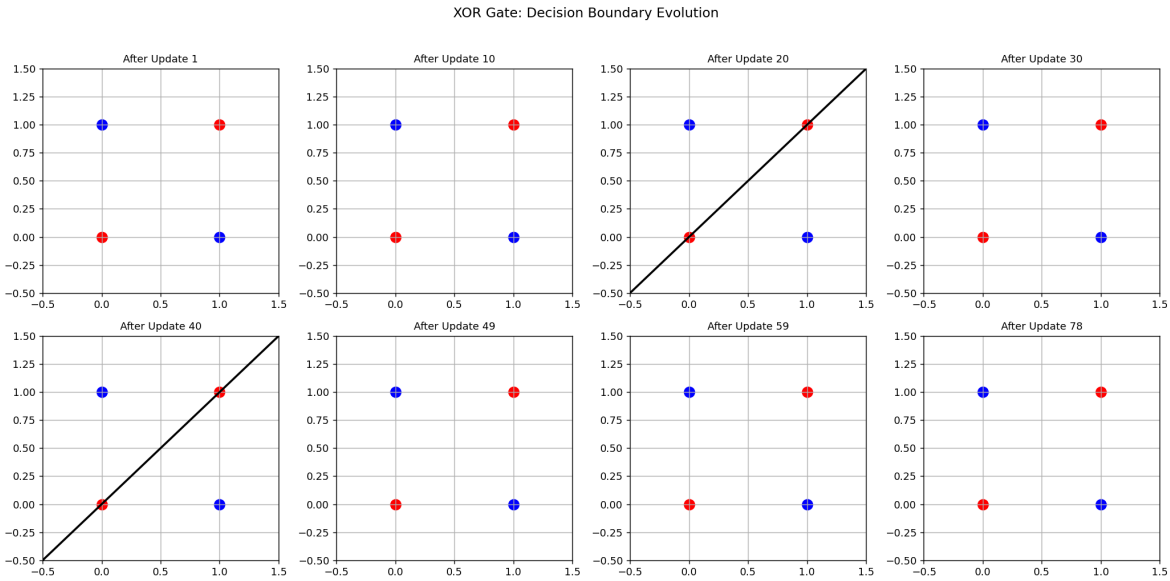

Figure 22: Decision boundary evolution for the XOR Gate across 20 epochs (78 weight updates).

Inference: *The boundary keeps swinging between the same few positions instead of settling, since no single straight line can separate the diagonal class pairs (0,0)/(1,1) from (0,1)/(1,0).*

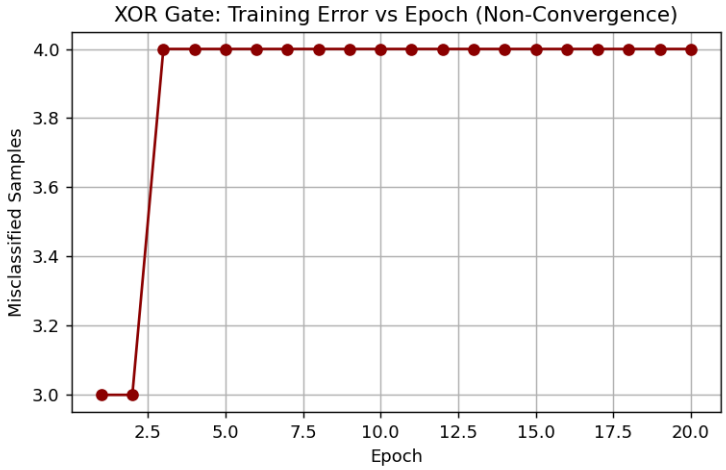

Figure 23: Training error (misclassified samples) versus epoch for the XOR Gate.

Inference: *Error gets permanently stuck at 3–4 misclassifications every epoch through epoch 20, confirming the single-layer perceptron never converges on XOR because the data is not linearly separable.*

9.2 Discussion Points (Answer in your own words)

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.

2. Compare the implementation with Scikit-learn's Perceptron.

3. Explain why a Single Layer Perceptron cannot solve the XOR problem.

4. Study the effect of feature normalization on model convergence.

10. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.